AgentCore as the Sandbox in Agentic RL

AgentCore Runtime as the execution sandbox, integrated with slime

Kaige Yang — Senior Applied Scientist, AWS

September 10, 2026

Outline

1. Agentic RL and the reference architecture
2. Foundations of AgentCore Runtime
3. Integrating AgentCore into slime
4. Tasks, environments, and dependencies
5. The episode and the code
6. Scaling and load-balancing
7. The demos
8. Results, resilience, and open problems

Section 1

Agentic RL and the reference architecture

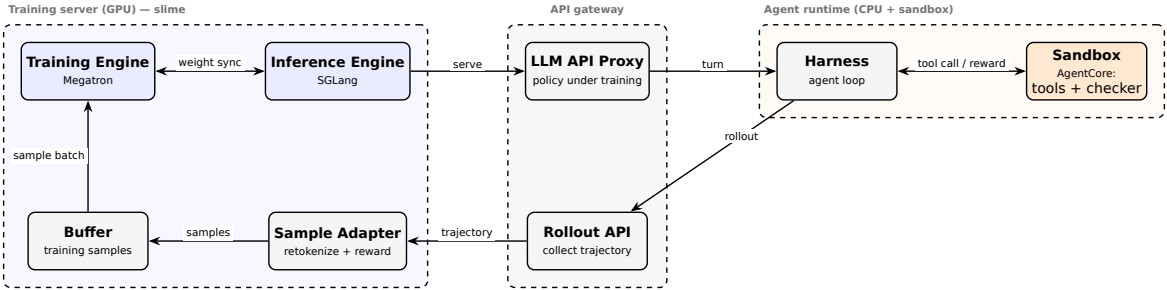
Purpose & roadmap — AgentCore as the sandbox for agentic RL

One idea: in agentic RL the policy acts over many turns against an executable environment, and a **programmatic checker** — not an LLM judge — turns the outcome into a scalar reward (RLVR).

Roadmap: reference architecture → AgentCore foundations (Runtime / session / microVM, compute types, lifecycle) → wiring into slime (call path, one hook, async rollout) → verifiable task families (and how one image serves many) → the episode & key code → scaling & load-balancing → all sixteen demos → results, resilience, open problems.

*Scope: AgentCore = the execution **sandbox**; **slime** = the trainer; the **harness stays in-process**.*

The reference architecture — one diagram



Generation lane (top, →); training-data lane (bottom, ←). ***AgentCore fills the sandbox seam.***

Five components, one job each — and the seam we fill

- **Trainer / Inference** (GPU) — learn, and generate turns at the *current* policy (kept fresh by **weight sync**).
- **Proxy** (CPU) — makes every model call hit the *policy under training*, not an external API.
- **Harness** (CPU) — the agent loop that owns one episode end-to-end.
- **Sandbox** (CPU/GPU) — the isolated box untrusted output runs in; the **checker scores the reward**.

AgentCore fills the sandbox seam — ad-hoc + isolated per episode. The harness needs *warmth*, not isolation, so it stays **in-process**. Training-data path: trajectory → reward attached → loss-masked sample → buffer → trainer step.

TITO — why the proxy exists

A harness works in **text**; RL trains on **token IDs**. Re-tokenizing decoded text can drift from what the model actually sampled — corrupting the action, the importance ratio, and the loss mask.

Token-in-token-out (TITO): the proxy records the **exact sampled `token_ids` + logprobs** at generation time and forwards them intact, so the trainer learns from the tokens that were really sampled.

- The proxy is the only component straddling both sides (text ↔ tokens), so it is the only correct place to capture them.
- For binary RLVR rewards this token-level exactness is not optional — slop silently corrupts the signal.

Section 2

Foundations of AgentCore Runtime

AgentCore basics — three concepts, precisely

- **Runtime** — a versioned, containerized host behind an ARN (`CreateAgentRuntime`). The deployable unit; holds no per-user state.
- **Session** — one episode, addressed by a `runtimeSessionId`. First invoke cold-starts compute; the same id re-routes to that warm compute.
- **microVM** — the unit of isolation: own kernel, CPU, memory, filesystem; sanitized on termination.

Cardinality is 1 : N : 1 — one Runtime serves many sessions; each session = exactly one microVM. **The number of Runtimes tracks distinct sandbox *images*, not tasks.**

Concurrency — episodes, sessions & the Runtime together

`N` concurrent episodes run as `N` session ids on ONE Runtime — not `N` Runtimes.

- The `generate` hook opens **one session per episode** (`CONC` in flight at once); each session id pins that episode to **its own microVM** (or its own GPU instance).
- Every tool op of an episode carries the *same* `runtimeSessionId`, so it returns to the **same warm microVM** — state persists within the episode, isolation holds across episodes.
- The capacity provider places those `N` sessions for you; **one image (one ARN)** backs them all. Add a **second ARN** only for a different image/substrate (e.g. GPU), then round-robin the pool.

So concurrency = more sessions, not more Runtimes.

Why a microVM (not just a container)

Running **untrusted, model-generated code** — isolation is the whole point: one episode's code can't see another's files, the trainer's weights, or the host, and the box is wiped at session end.

	Docker container	**AgentCore microVM**	E2B
Boundary	shared host kernel	**own kernel** (Firecracker-class)	own kernel
Runs your image?	yes	**your OCI image in a microVM**	Dockerfile templates
Managed by	you	**AWS, in your account** (IAM/VPC)	E2B service
GPU	host-dependent	**via Instances**	not core

Two compute types — CPU microVM vs GPU Instances

CPU work belongs on the microVM; Instances exist to add a GPU (EC2 in your account via a capacity provider). **No cross-session GPU time-slicing** — scale by adding sessions/instances, not by fractioning one (no MIG on L4).

Property	Serverless microVM	Instances (GPU)
GPU	none	g4dn/g5/g6/g6e… + inf2
Max session	8 h	14 d
Isolation	1 microVM / session	1 instance / session (**1 GPU**)
Filesystem	ephemeral	persistent EBS
Agents / session	1	many

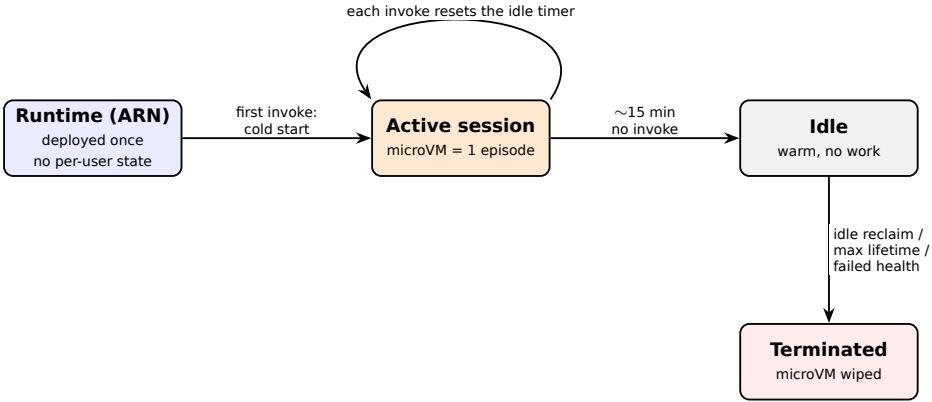
Building & deploying a Runtime — three ways, one contract

Contract: ARM64 image, serve `0.0.0.0:8080` with `/invocations` + `/ping`. Three ways to put your code on a Runtime (increasing control): the **`@aws/agentcore` CLI** (zero-Docker zip) → the **`bedrock-agentcore` SDK** (`@app.entrypoint` implements the contract) → **bring-your-own container** (max control — this project).

```
FROM public.ecr.aws/docker/library/python:3.12-slim   # CPU sandbox
RUN pip install --no-cache-dir bedrock-agentcore
COPY agentcore_slime_demo/sandbox_server_agent.py /app/
CMD ["python","-m","agentcore_slime_demo.sandbox_server_agent"]  # serves :8080
```

- **GPU image:** point `LD_LIBRARY_PATH=/usr/lib64` at the injected `libcuda.so.1`, and `pip install torch` **at `CMD`** (not baked) to stay under the **2048 MB** cap.
- **Deploy:** `deploy_two_runtime.py all` → ECR create → `buildx --platform` (arm64 microVM / amd64 GPU) → push → `CreateAgentRuntime` → (GPU) `CreateCapacityProvider` (`g6.xlarge`, VPC, `idleInstanceTimeout=600`) + attach.

Session lifecycle — launch, idle timer, terminate



*Launch is **implicit** (first invoke); terminate is **implicit** (idle/max-lifetime/health). A heartbeat ping resets the timer (`demo_cpu_quiesce`).*

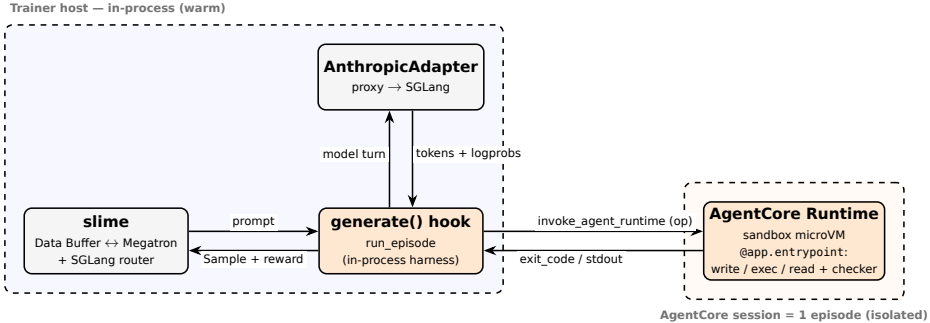
Lifecycle in the RL loop — row count ≠ instance count

- **One session per episode** → `N` concurrent episodes = `N` microVMs (or `N` GPUs).
- **A 100-row dataset does not launch 100 instances** — only `CONC` episodes are in flight, recycling through all rows over steps. What multiplies instances is **concurrency**, not data.
- **Launch/terminate are passive** — no explicit stop on the happy path; the weight-sync pause can trip the idle clock (→ resilience demos).
- **Latency: measure, don't assume** — a GPU first-invoke can take *minutes*; every op is instrumented (`sandbox_ms`, `sglang_gen_ms`).

Section 3

Integrating AgentCore into slime

How AgentCore sessions integrate into slime — the call path



*One `generate()` hook, one **`runtimeSessionId` per episode**; each tool call = one SigV4 `invoke_agent_runtime`.*

Plugging into slime — one hook, no fork

slime exposes the rollout as a **custom generation function**, so the agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path ⋯generate.generate` — drives the in-process harness; each tool call → one `invoke_agent_runtime` keyed by that episode's **`runtimeSessionId`**.
- **Design choice:** AgentCore is the **sandbox, not the harness**. The harness stays in-process (warm); the sandbox is the isolated per-session microVM. (It *can* host the harness too — kept as untested scaffolding.)
- The returned **loss-masked `Sample` + reward** flows straight back into slime's buffer — no fork of Megatron/SGLang.

slime — the trainer, and its asynchronous rollout

slime = Megatron (training) + SGLang (rollout), bridged by a **Data Buffer**; the agentic loop plugs into its rollout **generation hook** — no fork of the training kernel.

- **Sync mode** alternates train / rollout — each side's GPUs idle while the other runs.
- **Async driver** (`train_async.py`) overlaps them: Megatron trains batch k while the rollout side already generates batch $k{+}1$.
- **Fully-async rollout** (`fully_async_rollout`) keeps a *fixed pool of in-flight episodes across rollout boundaries*, so a step never waits on the slowest trajectory — the **long-tail** problem of agentic RL.
- **Why it matters here:** AgentCore sandboxes have **variable, sometimes long** per-episode latency; async rollout keeps the trainer fed while they run — at the cost of **staleness** (rollouts generated on an older policy version). This project runs the fully-async path (`MODE=B`).

Section 4

Tasks, environments, and dependencies

What tasks do frontier labs run — and how they score

Untrusted output runs in an isolated sandbox; a **programmatic verifier** produces the reward (Kimi K2: **>10,000 concurrent sandboxes**). **Two verifier styles:** *verifiable/rule-based* (binary, ungameable) vs *reference-based judge* (open-ended) — this project uses **RLVR only**.

Model	Verifier	Tasks	Reward
DeepSeek-R1	rule verifier + compiler	math, competitive code	tests pass / match
Qwen3	RLVR + live tool env	math, code, tool-use, RAG	rule + judge
Llama-Nemotron	LLM-judge	science, math, instruction	match + format
Kimi K2	simulator + real exec sandboxes	agentic tool use, code, SWE	tests + checks

Episode shape decides what you scale

Two axes set the config: **turns per episode** (`EPISODE_MAX_TURNS` + context) and **where the tokens are** (which side binds). **One microVM serves many families** — the sandbox is task-agnostic (`write_file`/exec/`read_file`); task specificity is *files seeded per episode*, not a new image.

Regime	Example	Bottleneck	Scale
single-turn, long trace	math, reasoning	**generation**	inference
few-turn, heavy checker	KernelBench (CUDA)	**sandbox**	sandbox / GPU
multi-turn, big observations	SWE, RAG, web	context growth	both + warm session

Sizing the config — the two knobs that follow from episode shape

- **`--rollout-max-response-len`** — the per-turn generation budget. Set it from the **p95 of your family's per-turn response** (math traces 1k–32k; raw code 100–800; CUDA kernel 200–1k; tool call 50–300).
  - *Too small* truncates the trace mid-thought → the checker scores a partial artifact **and** you train on the truncated sequence (biased gradient).
  - *Too large* wastes KV cache and slows decode (SGLang reserves capacity per sequence).
- **`EPISODE_MAX_TURNS`** (default 3) — caps generate→exec rounds; the **single biggest lever on episode cost** (each turn = one generation + one sandbox round-trip + a longer context).
  - single-turn (math, instruction) → 1; iterative code / kernel / SQL → 3–5; long-horizon SWE / web → 15–50+ (needs a **warm, long-lived session**).
  - more turns raise the reward ceiling but lengthen episodes → **more staleness** under async. Set per run in `run.sh`; no redeploy.

How AgentCore handles many tasks & many dependencies

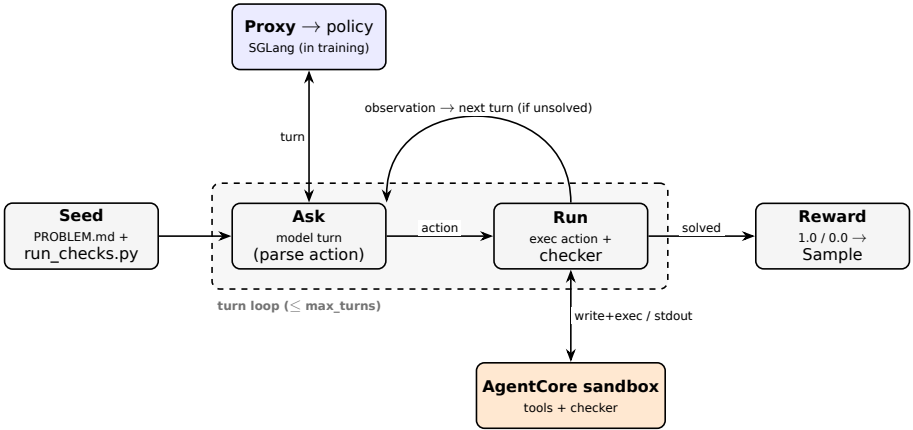
One microVM = one image per session. Task diversity is absorbed *inside* one generic image, in increasing order of how structural the difference is:

1. **Files, not images** — most CPU families (math `sympy`, code `unittest`, SQL `sqlite3`, tool-use, light CTF) share **one generic Python image**; the prompt, seed files, and hidden checker are written into *that episode's session*.
2. **Per-session install** — heavier deps go into the ephemeral workspace: **`CMD`-time** (implemented — the GPU image installs the ~2.7 GB torch wheel at boot to stay under the ~2 GB arm64 image cap) or **seed-time** `git clone`/`pip` (a *described* design, for SWE).
3. **Per-capability image** — only a genuine substrate difference (chiefly the **GPU** toolchain: `nvcc` + a real GPU) needs a **second Runtime ARN**. Isolation means one session's env never leaks into another's.

Section 5

The episode and the code

The episode, end to end — the loop



Seed → Ask (proxy) → Run (sandbox + checker) → Reward; unsolved loops back.

Step-by-step — one MBPP episode

1. **Deploy (once):** build the arm64 sandbox image → `CreateAgentRuntime` → get the ARN (the deploy driver, not slime).
2. **Launch run:** `run.sh` sets `SANDBOX=agentcore`, `FAMILY`, `CONC`, `EPISODE_MAX_TURNS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
3. **Open session:** slime hands a prompt to `generate()`, which rebuilds the `Task` and calls `make_sandbox("agentcore")` → **cold-starts** that episode's microVM (`runtimeSessionId`).
4. **Seed:** `write_file PROBLEM.md` + hidden `run_checks.py` into the session.
5. **Turn loop (≤ `EPISODE_MAX_TURNS`):** `policy.complete` via the **proxy** (TITO tokens) → extract `solution.py` → `exec("python3 run_checks.py")` in the sandbox → on failure, feed stdout back.
6. **Reward & learn:** `exit_code == 0` → **reward 1.0/0.0** from the in-sandbox checker → `build_token_stream` → loss-masked `Sample` → buffer → Megatron step → **weight sync** to SGLang.

Key code (1/2) — the in-process generate hook

slime calls this once per prompt: build the policy at the SGLang router, open **one sandbox per episode**, run the loop, return one loss-masked `Sample`.

```python
# agentcore_slime_demo/generate.py   (--custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    policy  = SGLangPolicy(router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_port}",
                           model_path=args.hf_checkpoint)
    task    = _task_from_sample(sample)            # rebuild Task from row metadata
    sandbox = make_sandbox(os.environ.get("SANDBOX", "agentcore"))
    async with sandbox as sb:                      # one session == one episode
        traj = await run_episode(policy, sb, task, sampling_params)
    tokens, resp_len, loss_mask, logprobs = build_token_stream(traj, tokenizer)
    return Sample(tokens=tokens, response_length=resp_len, reward=traj.reward,
                  loss_mask=loss_mask, rollout_log_probs=logprobs)
```

- Loss mask = **1 on the policy's own tokens, 0 on prompt / observation / tool tokens**.
- `build_token_stream` reuses the **recorded token_ids per turn** — no re-tokenization (TITO); returns a *bare* `Sample` (slime's async path chokes on nested lists).

Key code (2/2) — the sandbox client & the Runtime server

Each `write_file` / `exec` / `read_file` = **one SigV4 `invoke_agent_runtime`** keyed by the episode's session id; boto3 offloaded with `asyncio.to_thread` so `N` episodes run concurrently (with retries for transient blips + the GPU cold launch).

```python
# client — agentcore_runtime_sandbox.py
def _invoke_sync(self, payload):
    resp = self._client.invoke_agent_runtime(agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,        # >=33 chars; one per episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())

# server (deployed image) — the checker runs HERE, so the microVM is the reward source
app = BedrockAgentCoreApp()
@app.entrypoint
def entrypoint(request):        # dispatch on op: ping / write_file / read_file / exec
    return handle(request)
app.run()                       # serves 0.0.0.0:8080  (/invocations + /ping)
```

CPU → GPU is one env var (`SANDBOX=`) — zero change to the loop.

Who owns the lifecycle

slime's training kernel controls **neither** the Runtime nor the session.

- **Runtime** (create/delete the ARN) — the **deploy driver**, once before training. slime just gets the ARN.
- **Session** (per episode) — the `generate` hook's sandbox client: **launch implicit** (first invoke cold-starts the microVM); **terminate implicit** (idle reclaim, no explicit stop on the happy path).

Because termination is *passive*, the weight-sync pause can trip the idle clock and lose a warm session — the failure mode the resilience demos address.

Section 6

Scaling and load-balancing

The concrete setup — fix three components, vary the sandbox

On the 8×L40S box the demos **fix trainer, inference, and harness** and change only the sandbox ARN + task — the reference-architecture disaggregation made concrete (GPUs to train+infer, harness on cheap CPU, sandbox the isolated remote seam):

Component	Where	Config (shared)
Trainer	4 GPUs	Megatron, GRPO, `global_batch_size=32`, `ACTOR_GPUS=4`
Inference	4 GPUs	SGLang, 2 engines (2 GPU each), one **cache-aware router**, `ROLLOUT_GPUS=4`
Harness	in-process (CPU)	`run_episode`, fully-async `MODE=B`, `CONC=4`, `EPISODE_MAX_TURNS=3`
Sandbox	**remote AgentCore**	CPU microVM (arm64) **or** GPU g6 (amd64); one session/episode; `SANDBOX` selects it

Scaling the loop — the min() law

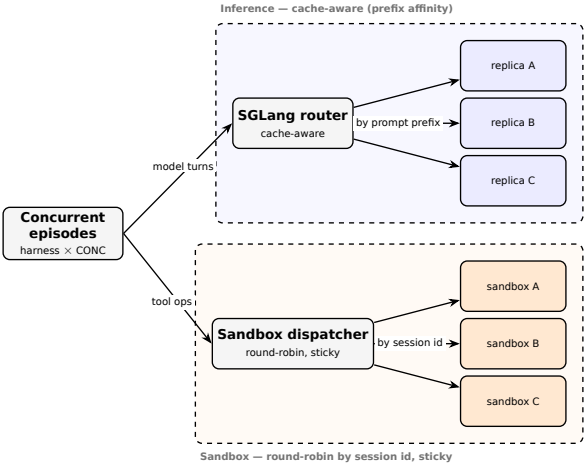
Rollout throughput = `min(inference, sandbox, harness)`. Scale two of three and the bottleneck just moves to the third.

First diagnose, then scale: compare `perf/sglang_gen_ms_*` vs `perf/sandbox_ms_*`.

- generation dominates ⇒ **inference-bound** → add SGLang replicas.
- sandbox dominates ⇒ **sandbox-bound** → add sessions / Runtimes / the GPU farm.

The harness is the easily-forgotten third dial — a persistent, warm process (GIL, `to_thread` pool, FDs). Scale it by **replicating warm workers** (`N × S`), not by making it ad-hoc.

Load-balancing differs by side — one diagram



Load-balancing differs by side — the rule

- **Inference — don't hand-roll.** Target slime's **cache-aware SGLang router**, which steers each request to the replica holding its prompt prefix. Naive round-robin destroys the prefix cache.
- **Sandbox — the session id *is* the balancer.** Within one Runtime the capacity provider places sessions; across a **pool of ARNs** add only **sticky round-robin by session id**, so every op of an episode returns to its warm microVM.

You never stand up a load balancer of your own.

Section 7

The demos

The demo set — 16 demos, one design

Policy always **Qwen3-4B**, harness **in-process**, AgentCore the **sandbox**; only the *substrate* and *task* vary.

- **6 canonical** — `cpu_single` (MBPP), `cpu_math`, `gpu_single` (KernelBench), `gpu_shared` (Ray farm), `cpu_quiesce` + `gpu_snapshot` (resilience).
- **8 per-task CPU fan-out** — one real HF dataset each: gsm8k, math500, deepscaler, humaneval, gretelsql, hotpotqa, xlam, mcp.
- **2 scale a new dimension** — `harness_scale`, `mixtask`.

Honest status: only `cpu_single` and `gpu_single` carry ~10-step curves; the rest are `NR=3` smoke or ready-to-run recipes.

All 16 demos (1/2) — canonical + resilience + math fan-out

Demo	Substrate	Task	Real use case	Status
`cpu_single`	CPU µVM	MBPP repair	test-driven coding	**trained** (10-step)
`cpu_math`	CPU µVM	math words	math post-training	smoke
`gpu_single`	GPU g6	KernelBench	CUDA kernel agent	**trained** (10-step)
`gpu_shared`	GPU g6 farm	KernelBench	GPU packing (scarce GPUs)	smoke passes e2e (local GPU)
`cpu_quiesce`	CPU µVM	heartbeat	keep sandbox warm over sync	offline
`gpu_snapshot`	GPU inst.	snapshot	survive instance recycle	offline + real recycle
`cpu_gsm8k`	CPU µVM	GSM8K	grade-school math	smoke, eval 0.91
`cpu_math500`	CPU µVM	MATH-500	competition math	smoke, eval 0.83

All 16 demos (2/2) — code / SQL / RAG / tool-use + scale

Demo	Substrate	Task	Real use case	Status
`cpu_deepscaler`	CPU µVM	DeepScaleR	scaled math reasoning	smoke, eval 0.70
`cpu_humaneval`	CPU µVM	HumanEval	function-synthesis coding	smoke, eval 0.81
`cpu_gretelsql`	CPU µVM	Gretel SQL	text-to-SQL analytics	smoke, eval 0.65
`cpu_hotpotqa`	CPU µVM	HotpotQA	multi-hop RAG / search	smoke, eval 0.21
`cpu_xlam`	CPU µVM	xLAM-60k	function-calling agent	smoke, eval 1.00
`cpu_mcp`	CPU µVM	ToolACE (MCP)	MCP tool-schema agent	smoke, eval 1.00
`cpu_harness_scale`	CPU µVM	DeepScaleR (fixed)	scale the rollout driver	offline sweep
`cpu_mixtask`	CPU µVM	math+SQL+CTF+tool	generalist multi-family stream	recipe only

Section 8

Results, resilience, and open problems

Measured results — the two regimes are real

Both bottleneck cases actually occur — from short (~10-step / smoke) runs, so read *shape and order-of-magnitude*, not converged science:

- MBPP: `eval/mbpp` 0.56 baseline; `step_time` ~23–78 s; **MFU 2–4 %** (goodput-vs-MFU gap, live).
- KernelBench: `wait_time_ratio` ~0.79–0.96 (trainer waits on the remote GPU checker); ~90 s torch install at cold start.

Demo	Sandbox op	Generation	Verdict
`cpu_single` (MBPP)	~0.12 s	~1.76 s	**generation-bound** → scale inference
`gpu_single` (KernelBench)	**~8.5 s** exec	~2.5 s	**sandbox-bound** → scale sandbox

GPU sharing — packing many episodes onto one GPU

KernelBench is sandbox-bound, and the isolated path needs *N whole g6 instances at once* — which stalls on a capacity-starved fleet. `demo_gpu_shared` packs `N` episodes onto **one** g6 instead (`SANDBOX=agentcore-ray-farm` — one line):

- All episodes invoke **one** GPU Runtime under a **shared `runtimeSessionId`**; inside, `_handle_farm` leases **one fractional-GPU Ray actor per episode** (keyed by a per-episode `sub_session`).
- `num_gpus=0.5` → two warm-CUDA actors share the L4; `0.25` → four. **No MIG on L4**, so isolation is software-only (per-process memory caps + an `nvcc` `flock` — a non-MIG L4 wedges under sustained compile storms).
- **The trade:** isolated = `N` clean instances (`N`× cost, unobtainable when the fleet is starved) vs shared = one instance, weaker isolation. **Primary motivation is capacity, not just cost.**
- **Status:** now runs **end-to-end on hardware** (`NR=3` smoke, `Job succeeded` + ckpt) via the local-GPU fallback; two fixes got it there — a **client-side open-concurrency gate** (cap opens at farm slots) + **synchronous checkpoint save** (seccomp blocks the async-ckpt `pidfd_getfd`). Fractional packing verified — no convergence curve yet.

What’s measured — and what’s only instrumented

Honest metric status across the demos — the last row is the gap: those keys exist (`docs/observability.md`) but are unpopulated, so no number is claimed for them.

Metric	Meaning	Status
`rollout/raw_reward`, `eval/<task>`	binary checker reward; held-out solve rate	**measured**
`perf/actor_train_tok_per_s`, `_tflops`	trainer throughput / achieved FLOPs	**measured**
`perf/mfu`	FLOPs util = TFLOPS ÷ 362 (L40S peak)	**measured** (2–4 %)
`perf/wait_time_ratio`	fraction of step the trainer waits on rollout	**measured**
`[sglang-gen]` / `[sbx-op]` p50/p95	per-turn generation / per-op sandbox latency	**measured** (drives scaling)
`perf/step_time`	per-step wall-clock (train + rollout)	**measured** (23–78 s CPU; 110–804 s GPU)
staleness, discard ratio, IS ratio (TIS)	off-policy & horizon signals	**instrumented only** — none recorded

Surviving stalls — the two resilience levers

- **Heartbeat (`cpu_quiesce`)** — during weight sync, generation stalls and idle sessions would be reclaimed. A `SessionKeeper` ping resets the idle clock, keeping the *same* warm session — **no cold start per round.** Lives *outside* the harness.
- **Snapshot (`gpu_snapshot`)** — a GPU instance can be recycled and come back blank (no VM/EBS snapshot on AgentCore). Serialize the workspace to a store you own (S3) and restore onto a fresh session — **application-level** snapshot. The A/B recycle was verified against a live local Runtime (`REAL_SNAPSHOT` mode), not only simulated.

Heartbeat prevents loss; snapshot recovers from it.

Open problems — infrastructure & systems

Frontier scale surfaces recurring systems problems (with the mitigations people ship):

- **Long-tail stragglers / partial rollouts** — generation is >90 % of RL runtime; cap-and-resume (Kimi), active partial rollouts (APRIL +22–44 %, now in slime), dynamic sampling (DAPO).
- **Weight-sync cost + staleness** — 100s of GB at 1T scale; fast transfer (Kimi ~20 s), zero-redundancy resharding (veRL), one-step-off async (AReaL ~2.77×). *This project accepts staleness (fully-async) — and doesn't record it yet.*
- **Goodput ≫ MFU** — interleaved bandwidth- vs compute-bound phases; colocate-with-offload (frontier default) vs disaggregated placement. *The 2–4 % MFU here is that gap.*
- **Scaling sandboxes to 10k+** — microVM isolation + managed backends. *Exactly AgentCore's per-session microVM; scale via a pool of ARNs + the Ray farm.*
- **Cache-aware routing / fault tolerance** — RadixAttention + SGLang router (~1.9×); heartbeats + sandbox recycling. → *the two resilience demos.*

Open problems — algorithms

- **Train/inference logprob mismatch** — same weights, different token probs (kernels / precision) → silently off-policy. **TITO** fixes the *tokenization* half; **TIS** (truncated importance sampling) the *numeric* half — instrumented-but-unrecorded here.
- **GRPO → variants** — entropy collapse, length bias, zero-gradient groups; drop-in upgrades **DAPO** (clip-higher, dynamic sampling), **Dr. GRPO**, **GSPO** (sequence-level, underpins Qwen3).
- **Long-horizon credit** — a sparse terminal reward mis-credits a multi-step episode; turn-level grouping (**GiGPO** +9–12 %). *Needed for SWE / web as `EPISODE_MAX_TURNS` grows.*
- **Reward hacking / verifier trust** — binary RLVR sidesteps neural-RM hacking, but a benchmark gain doesn't prove the checker did the work ("Spurious Rewards").
- **Entropy collapse; dynamic sampling & the discard ratio** — clip-higher / KL-Cov; keep only `0 < correct < G` groups. *`discard_ratio` is defined but 0 in current runs.*

This project's open work: record the off-policy signals (staleness, IS ratio, discard ratio) + add TIS; move GRPO → a variant as response length grows; turn-level credit for long-horizon families.

Summary

- **Can:** isolated per-session sandbox for untrusted output (CPU default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, **GPU on Instances**; a one-hook slime integration with the harness in-process.
- **Cannot:** a microVM has no GPU; no cross-session GPU fractioning (no MIG on L4); a session can't exceed 8 h without moving to Instances (14 d).
- **How to run:** build a sandbox image → deploy a Runtime (+ GPU capacity provider) → seed a task → `invoke_agent_runtime` per tool call → in-sandbox checker gives the reward → slime trains via one `generate` hook. **CPU↔GPU is one env var.**

One image, N sessions, one hook: AgentCore is the isolated per-episode sandbox; slime does the learning.